Desarrollo de aplicaciones seguras

MÁSTER UNIVERSITARIO EN SEGURIDAD INFORMÁTICA (CIBERSEGURIDAD)

Sara Balderas Díaz (sara.balderas@uca.es)
Gabriel Guerrero Contreras (gabriel.guerrero@uca.es)

CONSTRUCCIÓN DE APLICACIONES SEGURAS (PARTE 2)

Métricas (I)

Densidad de defectos:

$$densidad = \frac{número\ de\ bugs}{número\ de\ líneas\ de\ código}$$

Los **defectos:**

- se utilizan habitualmente como **medida de calidad;**
- son un referente erróneo de cuánto **riesgo plantea el código**.

Métricas (II)

Ejemplo 1:

```
1 /* This program computes Body Mass Index (BMI). */
2 int main(int argc, char** argv)
3 {
4    char heightString[12];
5    char weightString[12];
6    int height, weight;
7    float bmi;
8
9    printf("Enter your height in inches: ");
10   gets(heightString);
11   printf("Enter your weight in pounds: ");
12   gets(weightString);
13   height = atoi(heightString);
14   weight = atoi(weightString);
15   bmi = ((float)weight/((float)height*height)) * 703.0;
16
17   printf("\nBody mass index is %2.2f\n\n", bmi);
18 }
```

Métricas (III)

Ejemplo 2:

```
1 /* This program computes Body Mass Index (BMI). */
2 int main(int argc, char** argv)
3 {
4    int height, weight;
5    float bmi;
6
7    height = getNumber("Enter your height in inches");
8    weight = getNumber("Enter your weight in pounds");
9    bmi = ((float)weight/((float)height*height)) * 703.0;
10
11   printf("\nBody mass index is %2.2f\n\n", bmi);
12 }
13
14 int getNumber(char* prompt) {
15    char buf[12];
16    printf("%s: ", prompt);
17    return atoi(gets(buf));
18 }
```

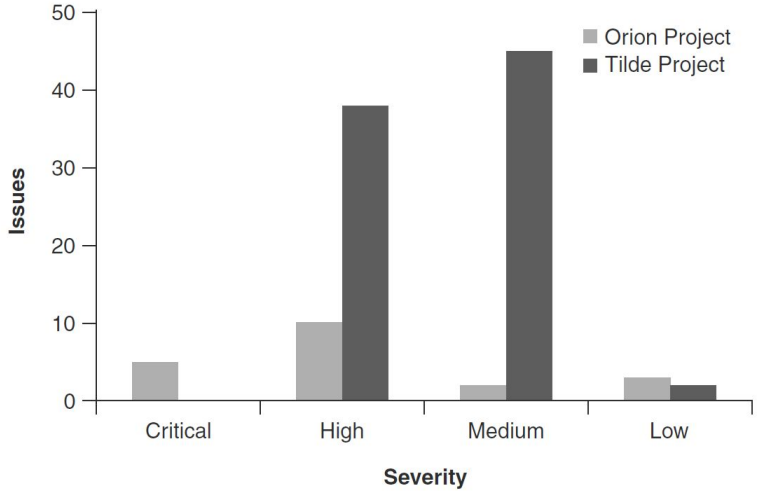
Métricas (IV)

Medición de la **densidad de la vulnerabilidad**:

- **Comparar la densidad de vulnerabilidad** entre diferentes módulos o proyectos ayuda a formular las **prioridades de revisión**.

Comparación de **proyectos por gravedad**:

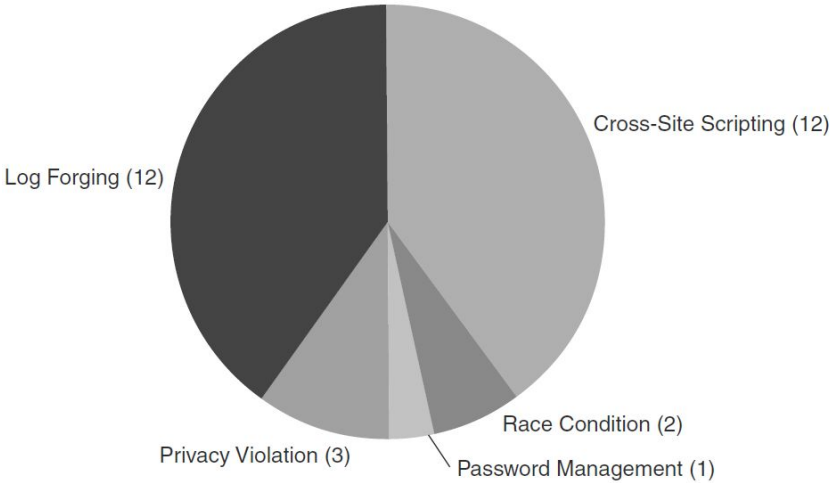
- Los **resultados del análisis estático** se utilizan para **comparar proyectos**.



Resultados del análisis del código fuente desglosados por gravedad para dos subproyectos (Chess et al. 2007)

Métricas (V)

Desglose de los **resultados por categoría**:

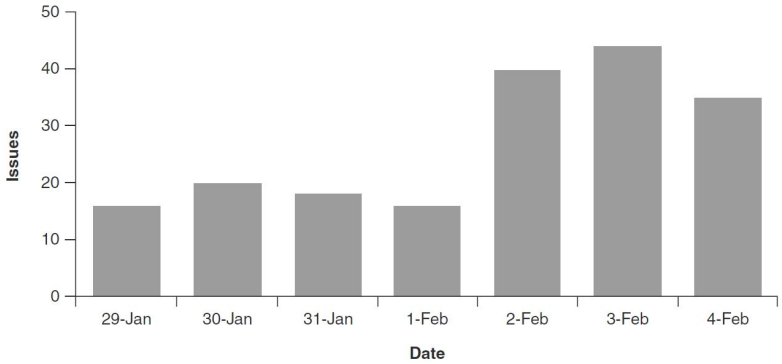


Resultados del análisis del código fuente desglosados por categorías (Chess et al. 2007)

Métricas (VI)

- Señalar **tendencias**:

  Ej., el número de problemas encontrados el 2 de febrero se dispara porque el grupo de desarrollo acaba de hacerse cargo de un módulo de un grupo que no se ha centrado en la seguridad.

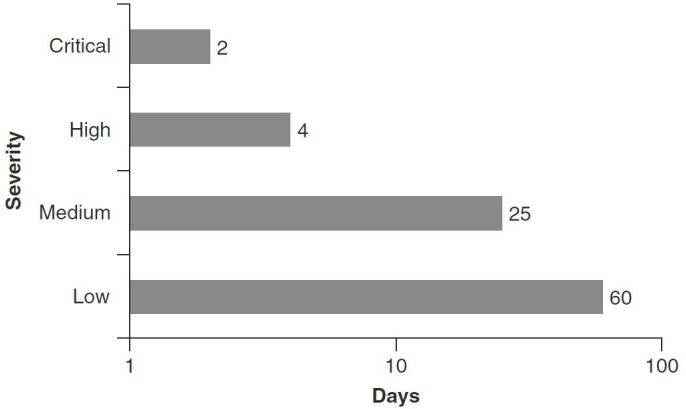


Resultados del análisis del código fuente reflejan el incremento de problemas a partir del 2 de febrero (Chess et al. 2007)

Métricas (VII)

- **Permanencia** de la vulnerabilidad:

  Después de que un auditor identifica una vulnerabilidad, ¿cuánto tiempo tardan, en promedio, los programadores en corregirla?



Permanencia de la vulnerabilidad en función de la gravedad (Chess et al. 2007)

CATEGORÍAS DE VULNERABILIDADES

Categorías de vulnerabilidades

Desglose de los **resultados por categoría**:

- Creación de registros (Log Forging)
- Secuencia de comandos en sitios cruzados (Cross-Site Scripting)
- Infracciones de privacidad (Privacy Violation)
- Condición de carrera (Race Condition)
- Gestión de contraseñas (Password Management)

Log injection y Log forging (I)

Las aplicaciones suelen utilizar los **archivos de registro** para almacenar un historial de eventos o transacciones para su posterior revisión, recopilación de estadísticas o depuración.

Log injection:

- **Inserción de datos** maliciosos o manipulados **en los registros de un sistema** con el objetivo de alterar su contenido o engañar a los sistemas de monitoreo y análisis de registros

Log forging:

- **Falsificación de registros** (creación de registros de registro falsos o maliciosos), a menudo desde cero, en lugar de simplemente modificar registros existentes que parecen reales.
- Introducir **entradas falsas con caracteres adecuados**.
- **Inutilizar el archivo** corrompiendo el formato a partir de **caracteres inesperados**.
- **Alterar las estadísticas** del archivo de registro.

Log injection y Log forging (II)

EJEMPLO

```
#código Python
logger.warning(f"Login fallido: {usuario}")
```

1. Log Injection: introducir contenido manipulado

```
ana
2026-10-28 INFO Login correcto: admin
```

La **inyección** consiste en **introducir el salto de línea (\n) y el texto adicional** en un campo que debería contener únicamente un nombre de usuario.

2. Log Forging: falsificar un evento

```
WARNING Login fallido: ana
2026-10-28 INFO Login correcto: admin
```

Por tanto, **Log Injection es el mecanismo utilizado para introducir contenido malicioso, mientras que Log Forging es la falsificación de un evento dentro del registro.**

Log injection y Log forging (III)

Ejemplo 1:

```
...
String val = request.getParameter("val");
try {
    int value = Integer.parseInt(val);
}
catch (NumberFormatException) {
    log.info("Failed to parse val = " + val);
}
...
```

Log injection y Log forging (IV)

Ejemplo 1:

```
...
String val = request.getParameter("val");
try {
    int value = Integer.parseInt(val);
}
catch (NumberFormatException) {
    log.info("Failed to parse val = " + val);
}
...
```

Si el usuario introduce la cadena "twenty-one" y el registro será:

```
INFO: Failed to parse val=twenty-one
```

Si un atacante introduce la cadena "twenty-one%0a%0aINFO:+User+admin+successfully+authenticated" y el registro será:

```
INFO: Failed to parse val=twenty-one
INFO: User admin successfully authenticated
```

Log injection y Log forging (V)

Ejemplo 2:

```
private final Logger logger
  = LoggerFactory.getLogger(LogForgingDemo.class);

public void addLog(String amount) {
    logger.info("Amount credited = {}" , amount );
}

public static void main(String[] args) {
    LogForgingDemo demo = new LogForgingDemo();
    demo.addLog("300");
}
```

Log injection y Log forging (VI)

Ejemplo 2:

```
private final Logger logger
  = LoggerFactory.getLogger(LogForgingDemo.class);

public void addLog(String amount) {
    logger.info("Amount credited = {}" , amount );
}

public static void main(String[] args) {
    LogForgingDemo demo = new LogForgingDemo();
    demo.addLog("300");
}
```

El **usuario realiza un pago** desde una aplicación web y el registro será:

```
web - 2017-04-12 17:45:29,978 [main]
  INFO  com.example.logforging.LogForgingDemo -
Amount credited = 300
```

Si un **atacante introduce la cadena** "\n\nweb – 2017-04-12 17:57:08,957 [main] INFO Amount reversed successfully", el registro será:

```
web - 2017-04-12 17:52:14,124 [main] INFO
com.example.logforging.
  LogForgingDemo - Amount credited = 300
web - 2017-04-12 17:57:08,957 [main] INFO
Amount reversed successfully
```

Cross site scripting (XSS)

Cross Site Scripting (XSS) pertenece a los ataques de inyección y consiste en inyectar *scripts* maliciosos, principalmente en sitios web.

Hay varios tipos de vulnerabilidades XSS:

- **Reflected Cross Site Scripting**
- **Stored Cross Site Scripting**
- **DOM-Based Cross Site Scripting**

Reflected Cross Site Scripting (I)

Reflected Cross Site Scripting se produce cuando un atacante **inyecta código ejecutable en el navegador dentro de una única respuesta HTTP.**

- El ataque inyectado:
  - no se almacena en la propia aplicación;
  - no es persistente; y
  - sólo afecta a los usuarios que abren un enlace malicioso o una página web de terceros.
- La **cadena de ataque se incluye como parte de la URI o de los parámetros HTTP**.
- Son el **tipo más frecuente de ataques XSS**.
- La carga útil del ataque se ejecuta a través de **una única solicitud y respuesta**, por tanto, también se conocen como XSS de primer orden o de tipo 1.

Reflected Cross Site Scripting (II)

- Los atacantes suelen aprovechar estas vulnerabilidades para:
  - **instalar registradores de claves**;
  - robar las *cookies* de la víctima;
  - realizar **robos del portapapeles**; y
  - **cambiar el contenido de la página** (p. Ej., enlaces de descarga).
- Una de las principales dificultades para **prevenir** las vulnerabilidades XSS es la **codificación adecuada de los caracteres**. Por ejemplo:
  - La aplicación web podría filtrar “<script>”
  - Pero no podría filtrar “%3cscript%3e”, que simplemente incluye otra codificación de etiquetas.

Reflected Cross Site Scripting (III)

Ejemplo 1:

Página web que nos da la bienvenida (Welcome %username%) y un enlace (*link*) de descarga.



- Cada punto de entrada de datos puede dar lugar a un ataque XSS.

Reflected Cross Site Scripting (III)

Ejemplo 1:

Página web que nos da la bienvenida (Welcome %username%) y un enlace (*link*) de descarga.



- Cada punto de entrada de datos puede dar lugar a un ataque XSS.

- Para comprobar si existe vulnerabilidad "jugamos" con la variable usuario:

`http://example.com/index.php?user=<script>alert(123)</script>`

- Si no se aplica ningún saneamiento (*sanitization*), puede aparecer la ventana emergente:



OWASP (2022)

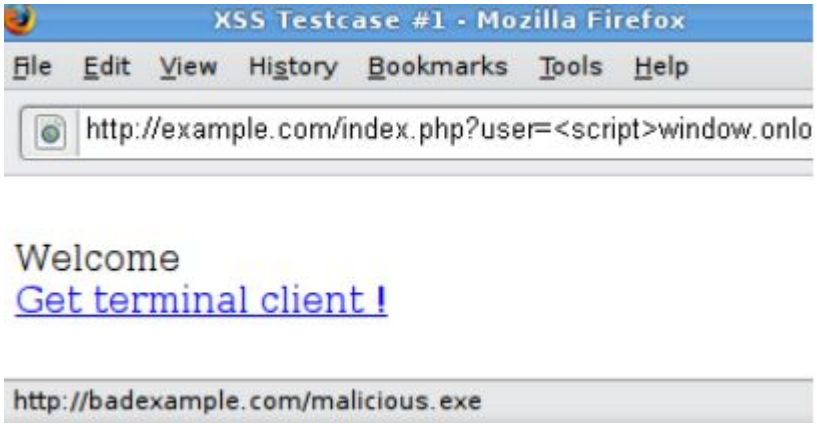
Reflected Cross Site Scripting (IV)

Ejemplo 2:

URL:

```
http://example.com/index.php?user=<script>window.onload = function() {
  var AllLinks=document.getElementsByTagName("a");
  AllLinks[0].href = "http://badexample.com/malicious.exe"; }</script>
```

Produce:



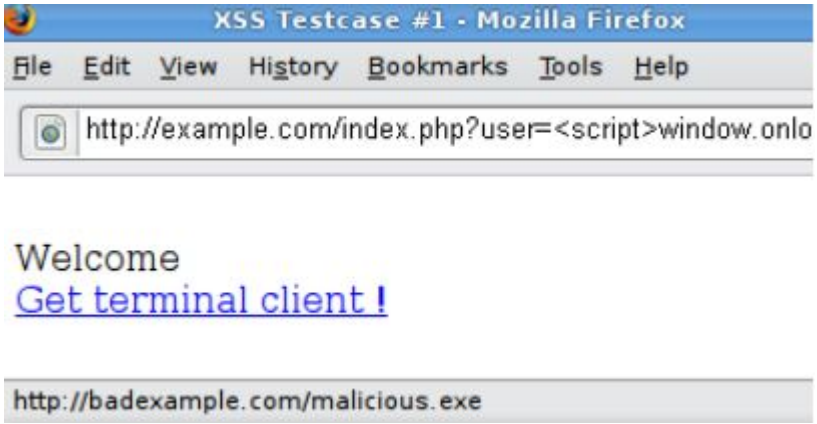
Reflected Cross Site Scripting (IV)

Ejemplo 2:

URL:

```
http://example.com/index.php?user=<script>window.onload = function() {
  var AllLinks=document.getElementsByTagName("a");
  AllLinks[0].href = "http://badexample.com/malicious.exe"; }</script>
```

Produce:



Vulnerabilidad

El archivo de descarga es malicioso

Reflected Cross Site Scripting (V)

Ejemplo 3:

Sintaxis o codificación diferente:

- `"><script >alert(document.cookie)</script >`
- `"><ScRiPt>alert(document.cookie)</ScRiPt>`
- `"%3cscript%3ealert(document.cookie)%3c/script%3e`

Puede producir:

- Que los filtros no lo detecten o sí, siendo estas variaciones toleradas por los navegadores.
- Robos de sesión de usuario.

Reflected Cross Site Scripting (VI)

Ejemplo 4:

A veces, el saneamiento, solo se aplica una vez y no de forma recursiva:

- `<scr<script>ipt>alert(document.cookie)</script>`

Por tanto, el atacante, puede superar el filtro enviando una cadena como esta que contenga múltiples intentos.

Stored Cross Site Scripting

Stored Cross Site Scripting es el tipo más peligroso de Cross Site Scripting.

Las aplicaciones web que permiten a los usuarios almacenar datos están potencialmente expuestas a este tipo de ataque.

Ejemplo:

- **Si la aplicación web permite la subida de archivos**, es importante comprobar si es posible subir contenido HTML.
- Si se permiten archivos HTML o TXT, **el payload XSS puede ser inyectado en el archivo subido**.
- Archivos de **apariencia inocua como JPG y GIF** pueden contener una **carga útil XSS que se ejecuta cuando son cargados por el navegador**.

Archivos de configuración

Ejemplo 1:

Entrada que espera el programa;

```
RewriteRule ^/img(.*) /var/www/img$1
```

La siguiente entrada provoca desbordamiento de búfer:

```
RewriteRule ^/img(.)(.)(.)(.)(.)(.)(.)(.)(.)(.*) /var/www/img$1$2$3$4$5$6$7$8$9$10
```

Vulnerabilidad

Infracciones de privacidad

El mal manejo de la información privada, como las contraseñas de los clientes o los números de la seguridad social, puede comprometer la privacidad de los usuarios, y a menudo es ilegal.

Los datos privados entran en el programa por:

- el usuario, que los introduce (p. ej., contraseña, información personal, etc.);
- estar almacenados en una base de datos u otro almacén de datos de la aplicación;
- una tercera parte.

Ejemplo:

```
pass = getPassword();
...
dbmsLog.println(id+":"+pass+":"+type+":"+tstamp);
```

- getPassword() devuelve la contraseña en texto plano suministrada por el usuario.
- Se registra una contraseña en texto plano en el sistema de archivos.

Gestión de contraseñas - Contraseña codificada

Credenciales hardcodeadas: aparece el usuario y contraseña de la base de datos directamente en el código fuente.

Ejemplo:

```
...
DriverManager.getConnection(url, "scott", "tiger");
...
```

SISTEMAS CRÍTICOS

Sistemas críticos

- Los **sistemas críticos** son aquellos de los que dependen las personas o las empresas.
- Su **mal funcionamiento puede causar importantes daños materiales o personales**.



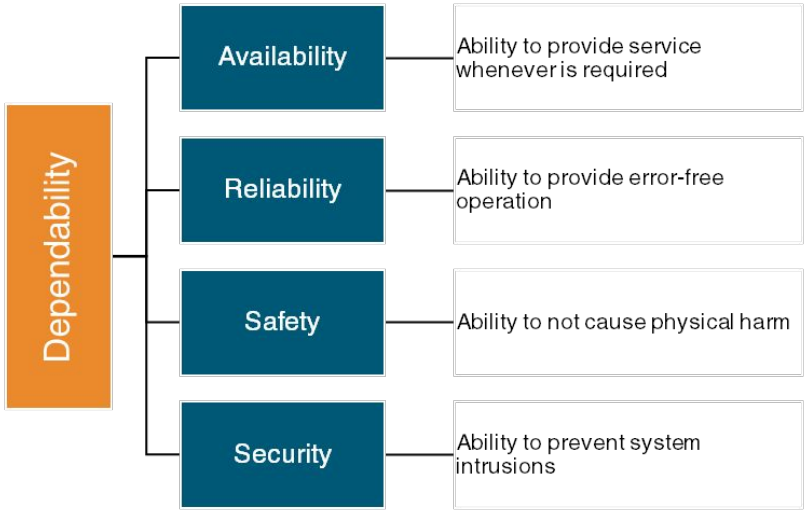
Tipos de sistemas críticos

En general, se clasifican en tres tipos:

- **Sistemas críticos de seguridad (*safety-critical systems*)**. Su mal funcionamiento puede causar daños humanos o medioambientales.
    - P. Ej., el sistema de control de una central nuclear.
- **Sistemas de misión crítica (*mission critical systems*)**. Su mal funcionamiento puede impedir la consecución de determinados objetivos.
    - P. Ej., el sistema de navegación de un avión.
- **Sistemas críticos para el negocio (*business critical systems*)**. Su mal funcionamiento puede suponer un elevado coste económico.
    - P. Ej., un sistema bancario en línea.

Propiedad de confiabilidad (I)

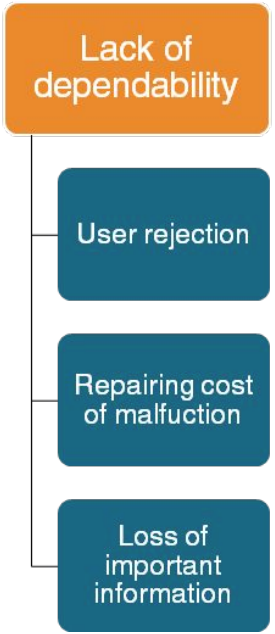
La propiedad de **confiabilidad (*dependability*)** es la más importante que deben cumplir estos sistemas.



Sommerville, I. (2005)

Propiedad de confiabilidad (II)

Las razones por las que esta propiedad es la más relevante pueden resumirse en tres principales.



Situaciones con sistemas críticos (I)

Un **sistema poco fiable o inseguro** es rechazado por los usuarios, lo que también afecta a la imagen de marca de la empresa.

- En abril de 2018, un cambio en la plataforma informática de TSB Bank hizo que los clientes no pudieran acceder a sus cuentas bancarias durante una semana.



TSB bank faces large compensation bill and fines over online chaos

MPs demand action as almost 2 million customers remain locked out of accounts

Latest updates in our live blog

Your key questions answered

In some TSB branches, staff were telling customers it could be another 48 hours before they could access their accounts fully. Photograph: Jill Mead/The Guardian

The Guardian, Wed 25 Apr 2018

Situaciones con sistemas críticos (II)

El **coste de una avería** puede superar en varias magnitudes el coste del propio sistema.

- En 2012, un fallo en el software de negociación de Knight Capital le hizo perder 460 millones de euros en 45 minutos.


Is Knight's $440 million glitch the costliest computer bug ever?

by Brian Patrick Eha @CNNTech

August 9, 2012: 10:22 AM ET

Knight Capital's computer bug cost the firm $440 million, making it one of history's most expensive software glitches.

CNN, August 9, 2012

Situaciones con sistemas críticos (III)

Un **sistema poco fiable** puede provocar la pérdida de información valiosa.

- En 2021, una actualización lanzada por Hewlett Packard Enterprise (HPE) en el superordenador de la Universidad de Kioto provocó la pérdida de 77 TiB de datos (34 millones de archivos afectados) de investigación científica.



The Stack, December 31, 2021

Disponibilidad *vs* fiabilidad (I)

La **disponibilidad (availability)** y la **fiabilidad (reliability)** son dos propiedades que generalmente se requieren en un sistema.

- Un sistema con **alta disponibilidad**, es decir, que siempre está disponible, pero con **baja fiabilidad**, es decir, que genera errores con frecuencia, **no será utilizable**.
- Un sistema con **alta fiabilidad**, es decir, que rara vez genera un error, pero con **baja disponibilidad**, es decir, que rara vez está disponible, **tampoco será utilizable**.





Disponibilidad *vs* fiabilidad (II)

Disponibilidad

Tiempo que el sistema está operativo y es capaz de prestar el servicio solicitado.

Dependerá de:

- La **frecuencia de los fallos** que hacen que el sistema no esté disponible.
- El **tiempo necesario para recuperarse** de esos fallos.

Fiabilidad

Probabilidad de que el sistema funcione sin errores según lo que se espera de él.

Los habrá de diferentes:

- **niveles de gravedad**,
- **contextos de ejecución**, y
- **tipos de usuarios**.

Fiabilidad (I)

Diferentes niveles de gravedad, p. ej., un fallo al realizar una transferencia bancaria online:

- Si el dinero no llega a la cuenta de destino, pero permanece en la cuenta de origen → Nivel de gravedad bajo
- Si el dinero no llega a la cuenta de destino, pero se descuenta de la cuenta de origen → Nivel de gravedad alto

Diferentes contextos de ejecución:

- P. Ej., los coches diseñados para climas templados pueden comportarse de forma diferente en países con climas más extremos (extremadamente fríos o calientes).

Diferentes tipos de usuarios:

- P. Ej., los sistemas abiertos al público pueden enfrentarse a comportamientos inesperados de los usuarios.

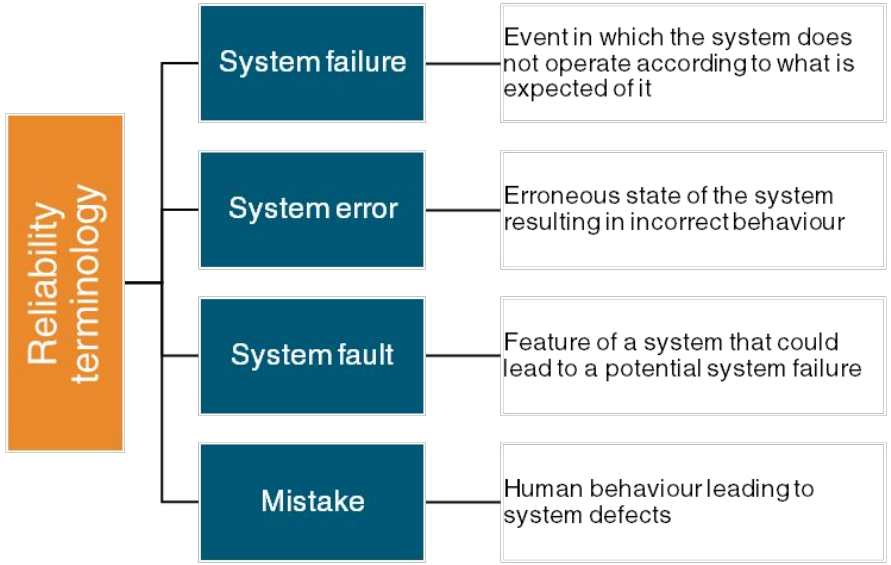
Fiabilidad (II)

- Es requerida una **especificación cuidadosa** que no deje a la libre interpretación el funcionamiento del sistema.
- Es importante tener en cuenta la **opinión de los expertos del sector o de los propios usuarios finales**.



Fiabilidad (III)

- Al analizar la fiabilidad de un sistema, se pueden distinguir **cuatro términos diferentes**.



Sommerville, I. (2005)

Fiabilidad (IV)

Hay **tres enfoques básicos** que ayudan a **mejorar la fiabilidad de un sistema en las diferentes fases de diseño y desarrollo**:

- **Evitar los enfoques propensos a los errores:** minimizar la probabilidad de que se cometan errores o de que éstos no se detecten.
    - P. Ej., lenguajes de programación propensos a errores, código repetitivo, no comprobar el tipo y el estado de los parámetros, etc.
- **Detección y eliminación de errores:** utilizar técnicas de verificación o validación cuando sea necesario.
    - P. Ej., pruebas sistemáticas o pruebas de caja blanca/negra.
- **Tolerancia a los fallos:** implementar técnicas que permitan al sistema recuperarse de ciertos fallos.
    - P. Ej., introducir la replicación de módulos o recursos que son críticos para el sistema.

Seguridad (*safety*) (I)

- Se define como la **propiedad por la que los programas informáticos no causan daños a las personas o a su entorno**.
- Es una **propiedad fundamental en los sistemas críticos relacionados con la salud, la industria o el ejército**.



Seguridad (*safety*) (II)

- El software de seguridad crítica puede dividirse en dos tipos principales:
  - **Software primario de seguridad crítica.**
    - El **software actúa como controlador principal del hardware**.
    - Un **fallo en el software provocará un mal funcionamiento del hardware y un daño potencial**.



P. Ej., el Therac-25, una máquina de radioterapia controlada por ordenador, aplicó entre 1985 y 1987 sobredosis masivas de radiación a los pacientes.

Seguridad (*safety*) (III)

- **Software secundario de seguridad crítica.**
  - El **software no controla directamente el hardware**, pero su mal funcionamiento puede causar daños a las personas o al medio ambiente de forma indirecta.
    - P. Ej., un error en una base de datos de medicamentos puede hacer que los médicos prescriban dosis erróneas de medicamentos a sus pacientes.

Contingencia: situación, que puede suceder o no, potencialmente peligrosa y que puede provocar un accidente.

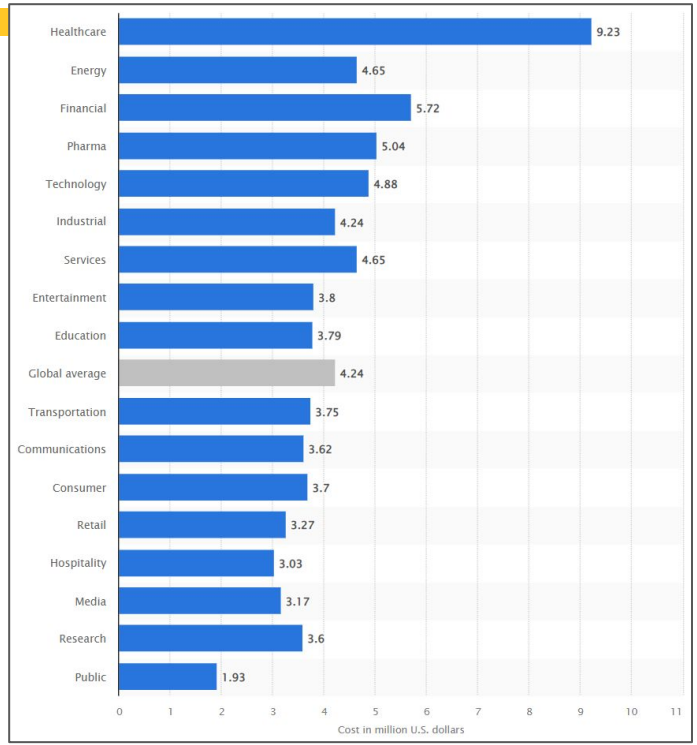
Seguridad (*safety*) (IV)

La seguridad de los sistemas puede gestionarse desde la fase de diseño:

- **Evitar contingencias.** El sistema se diseña para evitar situaciones potencialmente peligrosas.
    - P. Ej., el controlador de un horno microondas no permite que se ponga en marcha a menos que la puerta esté cerrada, para evitar fugas de radiación.
- **Detección y eliminación de contingencias**. El sistema está diseñado para detectar y eliminar las contingencias antes de que causen un accidente o un daño.
    - P. Ej., una estufa dejará de funcionar si detecta niveles elevados de CO2.
- **Limitación de daños.** Una vez producido un accidente, el sistema puede desplegar diferentes mecanismos para limitar los daños causados.
    - P. Ej., un sistema contra incendios en una sala de servidores, una vez detectado el fuego, desconectará la sala del suministro eléctrico para evitar que la situación se agrave.

Seguridad (*security*) (I)

- Se define como la **capacidad del sistema para resistir los ataques previos o accidentales que pueden dar lugar a intrusiones o acciones no deseadas en el sistema**.
- Las demás propiedades (disponibilidad, fiabilidad y seguridad) pueden verse comprometidas debido a acciones maliciosas.
    - P. Ej., El gasoducto transiberiano controlado por la Unión Soviética en 1982 fue atacado por el gobierno estadounidense mediante un programa malicioso (malware), provocando la explosión del gasoducto.*



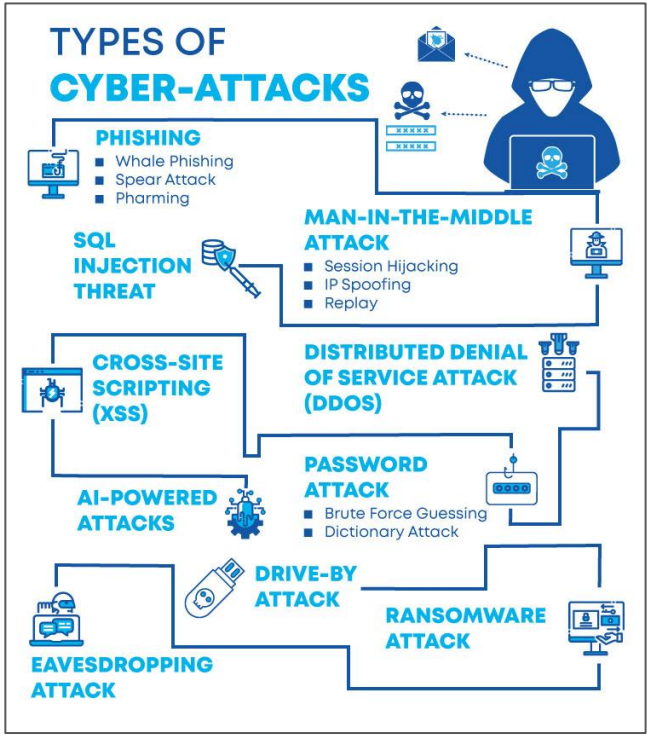
Average cost (million U.S. dollars) of data breaches, Statista

** The story is unconfirmed by the US government.*

Seguridad (*security*) (II)

Los ataques pueden clasificarse en tres grandes grupos:

- **Ataques de denegación de servicio (DoS)**: afecta a la disponibilidad del sistema, impidiendo que los usuarios accedan a él.
- **Corrupción de datos y/o programas**: los diferentes componentes del sistema pueden ser alterados de forma no autorizada.
- **Divulgación de información confidencial**: la información sensible gestionada por el sistema queda expuesta a personas no autorizadas.



Great Learning Team, Jun 16, 2020

¡GRACIAS POR VUESTRA ATENCIÓN!



Sara Balderas Díaz (sara.balderas@uca.es)
Gabriel Guerrero Contreras (gabriel.guerrero@uca.es)